Matjakt: An Integrated DevOps Workflow

Einar Harri – eharri@kth.se
Christopher Massi – cmassi@kth.se
DD2482 DevOps

1. Architecture and development process

Matjakt is a grocery-price comparison application built with React, TypeScript and Vite. Product search, shopping-cart calculations and account preferences depend on Supabase Auth and the PostgreSQL Data API, accessed directly from the browser. Matjakt was originally built as Christopher's thesis project. The project adds an integrated DevOps workflow connecting source changes to verification, infrastructure provisioning and deployment. An unprivileged nginx container serves the frontend on Google Cloud Run.

Separate Google Cloud and Supabase projects isolate the course environment from existing users. Local development and CI use a synthetic ICA and Coop catalog with known prices and geographic cases, making assertions reproducible. The hosted environment contains a catalog snapshot from the original application, excluding accounts, profiles, crawler history and Storage objects.

Code, migrations, tests, infrastructure and operating instructions are versioned together. Pinned runtime and dependency versions and a frozen lockfile constrain installation differences. The Docker-based Supabase CLI provides local database and authentication services from committed configuration and fixtures.

Pull requests run linting, TypeScript checks, unit and database tests, browser journeys and a production build. GitHub Actions checks Terraform formatting, while HCP Terraform validates infrastructure and produces speculative plans. An aggregate gate requires all mandatory checks to pass. Branch protection additionally requires an up-to-date branch, resolved discussions and review.

Following successful verification on main, the release workflow applies migrations and packages the verified frontend into an immutable container image. A tagged Cloud Run revision is deployed without live traffic and smoke-tested against the hosted backend. That same revision is then promoted and checked live. Release metadata and a version endpoint connect the deployed artifact to the source revision.

2. Infrastructure and security

GitHub Actions integrates verification and release with the existing development platform. Deployment explicitly depends on successful checks. Releases are serialized and reject obsolete main revisions, reducing the risk of an older workflow overtaking a newer change. Actions are pinned to commit SHAs, with Dependabot proposing updates.

Terraform manages Google Cloud APIs, Cloud Run configuration, Artifact Registry, service identities, Workload Identity Federation and budget notifications. HCP Terraform provides remote execution and state locking, with separate plan and apply identities.

Ownership is divided between provisioning and release: Terraform manages service configuration, while the release workflow controls the application image, revision and traffic. Excluding deployment fields from Terraform reconciliation prevents infrastructure applies from overwriting releases or reversing rollbacks.

Google Cloud access uses short-lived OIDC credentials rather than downloaded service-account keys. GitHub trust is restricted to the repository, main branch and release workflow; HCP trust is restricted to the designated workspace. Deployment permissions target the required resources. The runtime identity has no project roles, and Cloud Run scales to zero with at most two instances.

Because the browser accesses Supabase directly, SQL grants and row-level security enforce authorization. Profiles are restricted by authenticated user ID, and hosted email confirmation remains enabled. Migration credentials are confined to the release environment, where a guard checks the destination against the frontend's backend configuration.

The database baseline was reconstructed from application contracts and subsequently compared with a read-only schema export. Original search definitions were incorporated alongside adapted grants and profile authorization. PostGIS supports checkout distance calculations. Crawler infrastructure and obsolete functions are excluded.

3. Testing and verification

Verification covers application logic, database behaviour and browser interactions. Unit tests exercise quantities, discounts, fuel costs, geography and environment boundaries. Database tests check search, filtering, pagination, geographic selection and known prices. Profile-isolation assertions verify that accounts cannot access or modify other users' profiles.

Browser journeys cover search, details, cart calculations, login and profile persistence against real local Supabase services. Routing and map-tile requests are mocked to limit external variability. Empty results and an injected backend failure test error handling.

The complete local chain passed against a fresh database and clean source snapshot. An intentionally incorrect fuel-cost assertion also tested enforcement: the aggregate gate failed and release was prevented. Verification passed after the assertion was corrected.

Infrastructure verification included a speculative plan, automatic apply and subsequent no-change plan. The application release completed migrations, OIDC authentication, preview tests and live promotion. Both endpoints identified the same verified source revision.

Recovery testing redirected traffic to a previous Cloud Run revision, verified functionality and restored the current revision. Both revisions contained the same source artifact during this test, so it demonstrates traffic recovery rather than recovery from a defective change.

The hosted catalog added functional verification with approximately 89,600 products, 538 stores and 5.94 million prices. After setting the Supabase database to an appropriate disk size, the guarded transaction completed with matching counts, no orphan prices and unchanged profiles. Hosted search and checkout checks confirmed agreement between database prices and cart totals, without establishing production-scale performance.

4. Limitations and trade-offs

Preview and live share a database in order to save costs. Migrations can therefore affect live traffic before frontend promotion, making backward compatibility essential. Frontend rollback restores an earlier revision; database defects require forward corrective migrations.

Hosted Supabase creation remains a documented bootstrap step rather than a Terraform-managed resource. Migrations and local configuration are versioned, but hosted API and authentication settings require comparison with documented values.

Synthetic fixtures support deterministic assertions, while the hosted snapshot demonstrates functionality with realistic data. Neither establishes crawler reliability, continuous price accuracy or sustained-load performance. Catalog updates require refreshing the materialized deals view. Its intentional API exposure produces an advisor warning because the view lacks row-level security; the contents exclude profiles and contain public catalog data only.

Budget notifications reveal spending without stopping consumption. Supabase compute and retained images add costs beyond the Google Cloud budget. Retention and destruction protection support recovery but complicate cleanup. The frontend also reports a large bundle; code splitting remains outside scope. Dependency audits provide dated evidence and require continued maintenance.

5. AI assistance

OpenAI Codex assisted with design analysis, implementation, configuration, migrations, tests, verification, catalog import and documentation. Architectural decisions and manual review of generated code and configuration were performed by the project members. Automated checks supplemented that review. Responsibility remains with the authors.